

Final review

COMP 551 Applied machine learning

Yue Li

Assistant Professor
School of Computer Science
McGill University

April 15, 2025

Final exam coverage and format

- Time: 9-noon on April 16
- Location: McGill Sports Complex Field House 35-44

Final exam covers *everything* in Modules 1-7 with more emphasis on the post-midterm materials (i.e., Module 4.5 - Module 7).

1. You have 180 minutes to write the exam.
2. Hard-copy notes and books are allowed but electronics are not allowed.
3. This exam contains 25 questions on 18 pages for total of 100 points (with the last two pages as scribble sheet).
4. 18 multiple-choice questions (54 pts, 3 pts each): answer them on a scantron
5. 7 short answer questions (46 pts, 6+5+5+10+5+10+5): write your answer directly below each question.

Advice: Try not to spend too much time searching answers through your notes as it will slow you down and you will not have enough time to complete this exam in 3 hours.

Module 1. Evaluation of ML methods

Given a confusion table,

	Predicted negative	Predicted positive
Actual negative	$\#TN$	$\#FP$
Actual positive	$\#FN$	$\#TP$

know how to compute:

$$TPR = TP / (FN + TP)$$

$$FPR = FP / (TN + FP)$$

$$Precision = TP / (FP + TP)$$

$$Recall = TPR$$

- What values above do we need above to construct an ROC curve?
- What about Precision-recall curve?

TPR vs FPR

Choose all possible tasks (left) that each ML method (right) can do

1. regression

2. binary classification

3. multi-class classification

4. multi-label classification

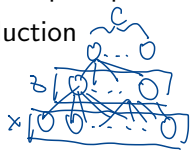
5. generate text given a prompt

6. dimensionality reduction

7. clustering

$$\hat{y}_c = \frac{\exp(xw_c)}{\sum_c \exp(xw_c)}$$

$$C \quad y_1, \dots, y_C$$



$$TCE = CE_1 + CE_2 + \dots + CE_C$$

$$\hat{y}_c = \sigma(xw_c), c \in \{1, \dots, C\}$$

$$y_1^{(n)} = 1, y_2^{(n)} = 1$$

• KNN (Module 2): 1-4

• Decision Tree (Module 3): 1-4

• Linear regression (Module 4.1):

• Logistic linear regression (Module 4.2):

• Multiclass linear regression (Module 4.3): 3

• MLP (Module 5.1 & 5.2): 1-4

• ConvNet or CNN (Module 5.3): 1-4

• BERT (Module 5.4) MLM 1-4 unaligned

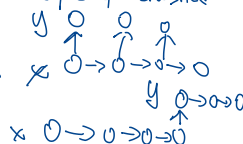
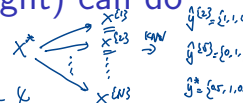
• RNN (Module 5.4) Seq2Vec, Seq2Seq aligned

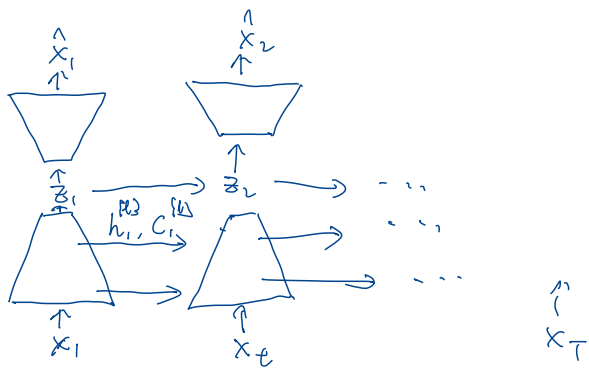
• GPT (Module 5.4) 1-5

• Random forest (Module 6) 1-4

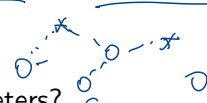
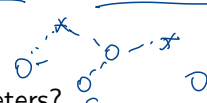
• K-means (Module 7) 7

• PCA (Module 7) 6



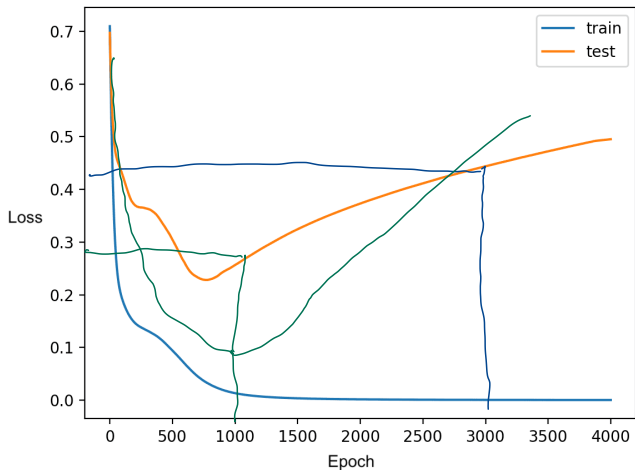


What are the hyperparameters of these ML methods, if any?

- KNN (Module 2): $k^{\uparrow}$, distance $\rightarrow V^{\downarrow}, B^{\uparrow}$ $k < N$
- Decision Tree (Module 3): tree depth $\uparrow \rightarrow V^{\uparrow}, B^{\downarrow}$
- Regularized linear regression (Module 4.5): $C = \|y - \hat{y}\|_2^2 + \pi \|w\|_1$, $\pi^{\uparrow} \rightarrow V^{\downarrow}, B^{\uparrow}$
- MLP (Module 5.1 & 5.2): width $\uparrow$, depth $\uparrow$, α , π , batch (SGD) $\rightarrow V^{\uparrow}, B^{\downarrow}$
- ConvNet (Module 5.3): # filters $\uparrow$, filter size $\uparrow$, stride, dilation, pooling size $\uparrow$
- BERT (Module 5.4): # attn heads $\uparrow$, embed. dim $\uparrow \rightarrow V^{\uparrow}, B^{\downarrow}$
- Random forest (Module 6): # trees $\uparrow$, tree depth per tree $\uparrow$ 67% $\rightarrow V^{\downarrow}, B^{\downarrow}$
- K-means (Module 7): k , distance  
- How do you choose the hyperparameters?  Supervisor: validation; unsupervised: BIC
- How does changing the hyperparameters change variance and bias (Module 4.5)?
Recall: the more complex the model the higher the variance and the lower the bias

Training vs testing loss

Student X wrote in assignment 3: “the misclassification rate on the test set for my model is 0.2 based on the curve below.” What’s wrong with it?



Module 4.5. Regularization

Module 4.5 Weight penalties

- Squared ℓ_2 penalty (Ridge): $\lambda \|\mathbf{w}\|_2^2 = \lambda \sum_d w_d^2 \propto -\log \mathcal{N}(\mathbf{w}; \mu = 0, \sigma^2 = 1/\lambda)$
- ℓ_1 penalty (LASSO): $\lambda \|\mathbf{w}\|_1 = \lambda \sum_d |w_d| \propto -\log \text{Lap}(\mathbf{w}; \mu = 0, b = 1/\lambda)$

Ridge fitting algorithm

The fitting algorithms (assuming $\mathbf{y}$ and $\mathbf{x}$ are centered):

- closed-form update: $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$
- gradient descent: $\nabla \mathbf{w}: \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}} (\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2) = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}$

Maximum a posteriori (MAP)

$$\mathbf{w}^* \leftarrow \arg \max_{\mathbf{w}} \log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) + \log p(\mathbf{w})$$

Handwritten notes below the equation:

- $\log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) \propto -\log \mathcal{N}(\mathbf{y}; \mathbf{X}\mathbf{w}, \sigma_y^2)$
- $\log p(\mathbf{w}) \propto -\log \prod_{\alpha} \mathcal{N}(w_{\alpha}; 0, 1/\lambda)$
- $\text{MCE} \propto -\log \prod_{\alpha} \mathcal{N}(w_{\alpha}; 0, 1/\lambda)$

LASSO coordinate descent algorithm to fit MAP estimates

Let $\hat{\mathbf{y}}_{\setminus d} = \mathbf{X}_{\setminus d} \hat{\mathbf{w}}_{\setminus d}$

$$\hat{\mathbf{y}} = \mathbf{X}_d \mathbf{w}_d + \sum_{d' \neq d} \mathbf{x}_{d'} \hat{\mathbf{w}}_{d'}$$

$$= \mathbf{X}_d \mathbf{w}_d + \hat{\mathbf{y}}_{\setminus d}$$

Algorithm 1 LASSO.fit($\mathbf{X}$, $\mathbf{y}$, $\lambda = 0.01$, max_iter=10⁵)

1: **while** not converged & niter $\leq$ max_iter **do**

2: **for** $d = 1 \dots M$ **do**

3:

$$\hat{\mathbf{w}}_d = \begin{cases} \frac{\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + \lambda}{\|\mathbf{x}_d\|_2^2} & \mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) < -\lambda \\ 0 & \mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) \in [-\lambda, \lambda] \\ \frac{\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) - \lambda}{\|\mathbf{x}_d\|_2^2} & \mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) > \lambda \end{cases}$$

4: **end for**

5: **end while**

$$\hat{\mathbf{y}}_{\setminus d} = \mathbf{X}_{\setminus d} \hat{\mathbf{w}}_{\setminus d}$$

$$= \sum_{d' \neq d} \mathbf{x}_{d'} \hat{\mathbf{w}}_{d'}$$

$$\hat{\mathbf{y}} = \phi(\mathbf{X}_d \mathbf{w}_d + \hat{\mathbf{y}}_{\setminus d}^*)$$

$$\neq \phi(\mathbf{X}_d \mathbf{w}_d) + \phi(\hat{\mathbf{y}}_{\setminus d}^*)$$

Questions: Suppose $\hat{\mathbf{y}} = \phi(\mathbf{X}\mathbf{w})$, where $\phi(x)$ is a non-linear function, does the above coordinate descent still work?

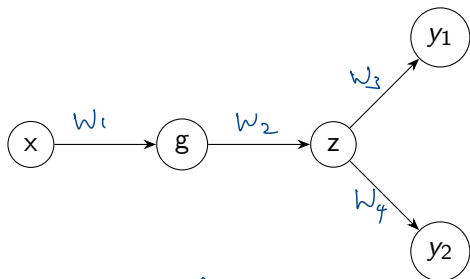
$$\hat{y} = \sigma(xw) = \frac{1}{1 + \exp(-xw)}$$

$$= \frac{1}{1 + \exp(-(x_d w_d + \sum_{d' \neq d} x_{d'} w_{d'}))}$$

$$\neq \frac{1}{1 + \exp(-x_d w_d)} + \frac{1}{1 + \exp(-\sum_{d' \neq d} x_{d'} w_{d'})}$$

Module 5.2 backpropagation

Derive the gradient of $x \rightarrow g$ weight for a multi-variate 1-layer MLP



$$a_x = xw_1, \quad g = \max(0, a_x)$$

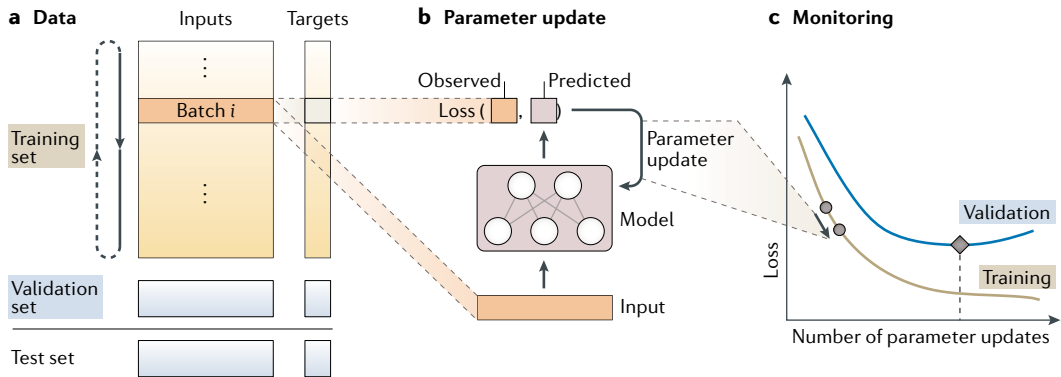
$$a_g = gw_2, \quad z = \sigma(a_g)$$

$$\hat{y}_1 = zw_3, \quad \hat{y}_2 = zw_4$$

$$L = (y - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2$$

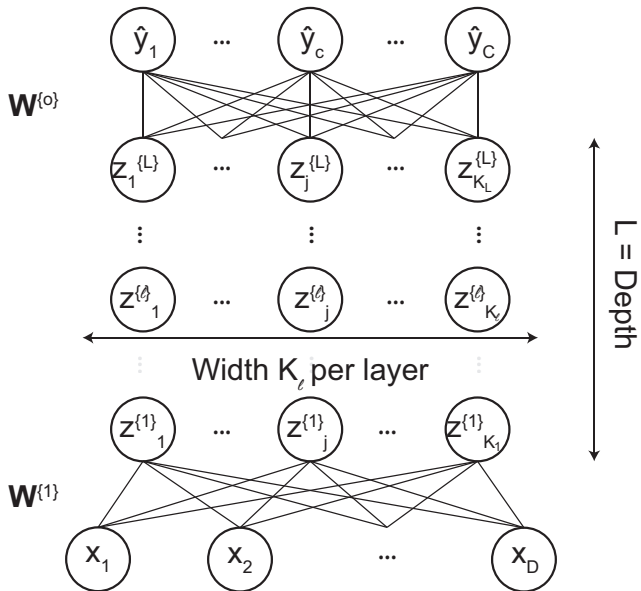
$$\begin{aligned} \frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial z} \frac{\partial z}{\partial a_g} \frac{\partial a_g}{\partial g} \frac{\partial g}{\partial a_x} \frac{\partial a_x}{\partial w_1} + \frac{\partial L}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial z} \dots \dots \dots \\ &= \left(\frac{\partial L}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial z} + \frac{\partial L}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial z} \right) \frac{\partial z}{\partial a_g} \frac{\partial a_g}{\partial g} \frac{\partial g}{\partial a_x} \frac{\partial a_x}{\partial w_1} \\ &= (-2(y - \hat{y}_1)w_3 - 2(y_2 - \hat{y}_2)w_4) z(1-z)w_2 I[a_x > 0] x \end{aligned}$$

Training neural networks via stochastic gradient descent

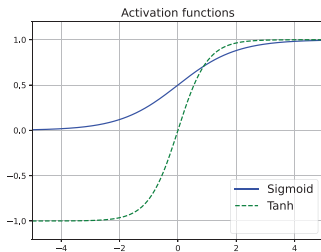
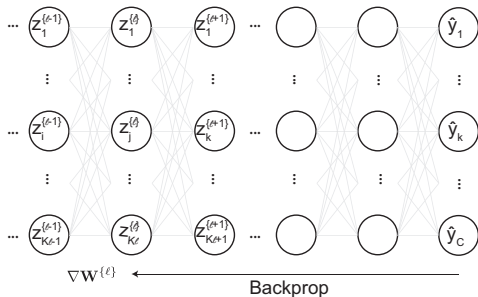


One epoch means go through all minibatches once (i.e., going through all N data points once). The number of parameter updates per epoch is the same as the number of minibatches.

L -layer MLP



Vanishing gradient caused by multiplying small gradients during backprop



$$\frac{\partial L}{\partial w^{(l)}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z^{(L)}} \frac{\partial z^{(L)}}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L-1)}} \frac{\partial z^{(L-1)}}{\partial a^{(L-1)}} \frac{\partial a^{(L-1)}}{\partial z^{(L-2)}} \cdots \frac{\partial z^{(l+1)}}{\partial a^{(l+1)}} \frac{\partial a^{(l+1)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial w^{(l)}}$$

For sigmoid:

$$z = \sigma(a) = \frac{1}{1 + \exp(-a)}$$

$$\frac{\partial z}{\partial a} = z(1 - z) \in [0, 1]$$

For hyperbolic tangent:

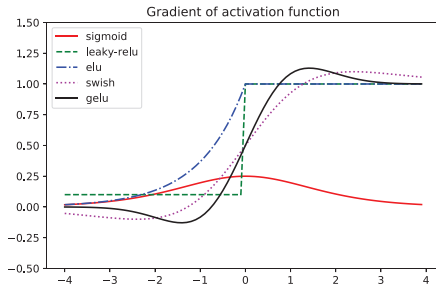
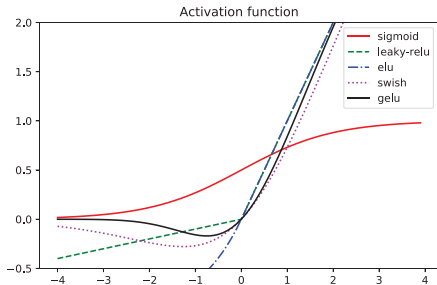
$$z = \tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)}$$

$$\frac{\partial z}{\partial a} = 1 - \tanh^2(a) \in [0, 1]$$

Non-saturating activation functions to reduce VG (Murphy22 13.4.3)

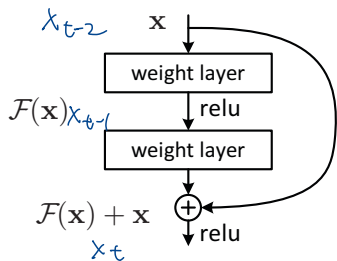
Name	Definition	Range	Reference
Sigmoid	$\sigma(a) = \frac{1}{1 + e^{-a}}$	$[0, 1]$	
Hyperbolic tangent	$\tanh(a) = 2 \sigma(2a) - 1$	$[-1, 1]$	
Softplus	$\sigma_+(a) = \log(1 + e^a)$	$[0, \infty]$	[GBB11]
Rectified linear unit	$\text{ReLU}(a) = \max(a, 0)$	$[0, \infty]$	[GBB11; KSH12]
Leaky ReLU	$\max(a, 0) + \alpha \min(a, 0)$	$[-\infty, \infty]$	[MHN13]
Exponential linear unit	$\max(a, 0) + \min(\alpha(e^a - 1), 0)$	$[-\infty, \infty]$	[CUH16]
Swish	$a\sigma(a)$	$[-\infty, \infty]$	[RZL17]
GELU	$a\Phi(a)$	$[-\infty, \infty]$	[HG16]

Can you compute derivative for these act functions?



Residual connection mitigates vanishing gradient

Residual connection allows gradient to flow from the output layer to the early layer.



The residual connection allows gradient at layer ℓ directly depend on the gradient at layer L regardless of the network depth.

The activation at the output layer with residual connection from layer ℓ :

$$\mathbf{z}^{\{L\}} = \mathbf{z}^{\{\ell\}} + \phi(\mathbf{z}^{\{L-1\}}\mathbf{W}^{\{L\}}) = \mathbf{z}^{\{\ell\}} + \tilde{\mathbf{z}}^{\{L\}}$$

Gradient for weights at layer ℓ :

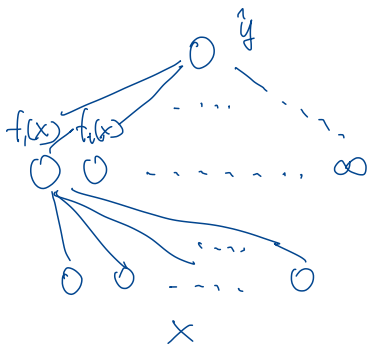
$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{\{\ell\}}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{\ell\}}} \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{L\}}} \frac{\partial \mathbf{z}^{\{L\}}}{\partial \mathbf{z}^{\{\ell\}}} \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{L\}}} \left(1 + \frac{\partial \tilde{\mathbf{z}}^{\{L\}}}{\partial \mathbf{z}^{\{\ell\}}} \right) \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{L\}}} \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} + \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{L\}}} \frac{\partial \tilde{\mathbf{z}}^{\{L\}}}{\partial \mathbf{z}^{\{\ell\}}} \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} \end{aligned}$$

Module 5.2 Bayesian optimization for choosing hyperparameters

Basic idea: Use a flexible regression model, i.e., Gaussian Process (GP) to **predict** the prediction error $J(\omega)$ as a function of hyperparameters ω (e.g., number of hidden units, number of hidden layers, weight penalty, initial learning rate, momentum, etc):

1. Train a set of neural networks based on an initial set of hyperparameter setting to produce $\mathcal{S} = \{\omega^{(s)}, J(\omega)^{(s)}\}_{s=1}^S$ on the validation data
2. Fit a GP using $\mathcal{S} = \{\omega^{(s)}, J(\omega)^{(s)}\}_{s=1}^S$ as the training set
3. Run GP to evaluate on all of the hyperparameter settings Ω
4. Choose the best setting $\omega^* \in \Omega$ with the lowest expected error, train the MLP with the hyperparameter setting ω^* and evaluate it on the validation set to get $J(\omega^*)$, and add the pair to the existing set $\mathcal{S}^* \leftarrow \mathcal{S} \cup \{\omega^*, J(\omega)^*\}$
5. Repeat 2-4 with the new training set $\mathcal{S} = \mathcal{S}^*$

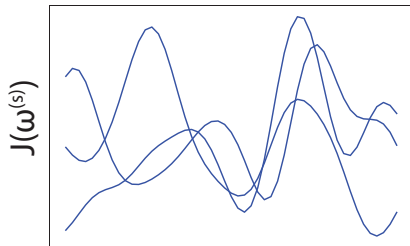
Reference: Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical bayesian optimization of machine learning algorithms. Advances in neural information processing systems, 25.



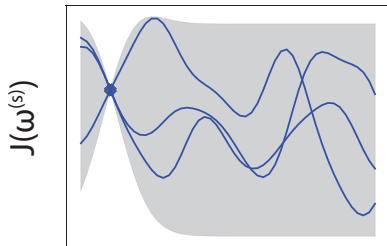
$$\hat{y} = \underbrace{\int f(x^*) p(f(x^*) | D^{\text{train}}) df}_{= \mathcal{N}(\hat{y}, \sigma_{x^*}^2)}$$

GP fitting on $\mathcal{S} = \{\omega^{(s)}, J(\omega)^{(s)}\}_{s=1}^S$ (source & Murphy22 Fig. 17.7)

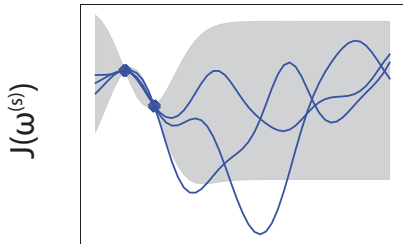
S=0



S=1

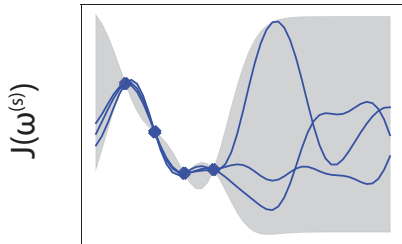


S=2



$\omega^{(s)}$

S=4



$\omega^{(s)}$

Illustration of 2D convolution

$\mathbf{W} \in \mathbb{R}^{H \times W}$ denotes a 2D filter. Convolution at the 0-based (and 1-based) pixel coordinate $[i, j]$ is:

$$[\mathbf{W} \circledast \mathbf{X}][i, j] = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{u,v} x_{i+u, j+v} \quad \left([\mathbf{W} \circledast \mathbf{X}][i, j] = \sum_{u=1}^H \sum_{v=1}^W w_{u,v} x_{i+u-1, j+v-1} \right)$$

For example, convolving a 3×3 input “image” $\mathbf{X}$ with a 2×2 filter “template” $\mathbf{W}$ to compute a 2×2 output (indices are 1-based in the convolution below):

$$\begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix} \circledast \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^2 \sum_{j=1}^2 w_{i,j} x_{i,j} & \sum_i \sum_j w_{i,j} x_{i,j+1} \\ \sum_i \sum_j w_{i,j} x_{i+1,j} & \sum_i \sum_j w_{i,j} x_{i+1,j+1} \end{bmatrix}$$

$$\begin{bmatrix} w_{1,1}x_{1,1} + w_{1,2}x_{1,2} + w_{2,1}x_{2,1} + w_{2,2}x_{2,2} & w_{1,1}x_{1,2} + w_{1,2}x_{1,3} + w_{2,1}x_{2,2} + w_{2,2}x_{2,3} \\ w_{1,1}x_{2,1} + w_{1,2}x_{2,2} + w_{2,1}x_{3,1} + w_{2,2}x_{3,2} & w_{1,1}x_{2,2} + w_{1,2}x_{2,3} + w_{2,1}x_{3,2} + w_{2,2}x_{3,3} \end{bmatrix}$$

Input		Filter		Output																	
<table><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	$\otimes$	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Module 5.3 Equivalent matrix multiplication form of convolution

$$\begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix} \circledast \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \end{bmatrix} =$$

$$\begin{bmatrix} w_{1,1}x_{1,1} + w_{1,2}x_{1,2} + w_{2,1}x_{2,1} + w_{2,2}x_{2,2} & w_{1,1}x_{1,2} + w_{1,2}x_{1,3} + w_{2,1}x_{2,2} + w_{2,2}x_{2,3} \\ w_{1,1}x_{2,1} + w_{1,2}x_{2,2} + w_{2,1}x_{3,1} + w_{2,2}x_{3,2} & w_{1,1}x_{2,2} + w_{1,2}x_{2,3} + w_{2,1}x_{3,2} + w_{2,2}x_{3,3} \end{bmatrix}$$

can be rewritten as:

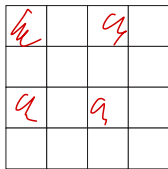
$$\begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{2,1} & x_{2,2} & x_{2,3} & x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix} \begin{bmatrix} w_{1,1} & 0 & 0 & 0 \\ w_{1,2} & w_{1,1} & 0 & 0 \\ 0 & w_{1,2} & 0 & 0 \\ w_{2,1} & 0 & w_{1,1} & 0 \\ w_{2,2} & w_{2,1} & w_{1,2} & w_{1,1} \\ 0 & w_{2,2} & 0 & w_{1,2} \\ 0 & 0 & w_{2,1} & 0 \\ 0 & 0 & w_{2,2} & w_{2,1} \\ 0 & 0 & 0 & w_{2,2} \end{bmatrix}$$

We can recover the 2×2 output by reshaping the 4×1 vector into 2×2

Module 5.3 Receptive field of ConvNet

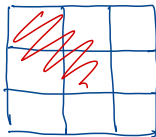
Using a 2×2 convolution filter with stride 1 or 2 to convolve a 4×4 input image, what's the receptive field of every pixel in the feature map at convolution layer 2?

Input image



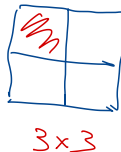
Convolution
→
Filter size: 2×2 ; Stride 1

Feature map at layer 1

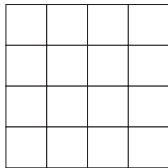


Convolution
→
Filter size: 2×2 ; Stride 1

Feature map at layer 2

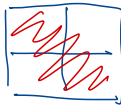


Input image



Convolution
→
Filter size: 2×2 ; Stride 2

Feature map at layer 1



Convolution
→
Filter size: 2×2 ; Stride 2

Feature map at layer 2



Module 5.4 RNN



The above RNN takes a 1×3 input vector $\mathbf{x}$ and computes a 1×2 hidden state $\mathbf{h}_t$ for $T = 9$ time points before making a prediction of a scalar output y :

$$\mathbf{h}_1 = \phi(\mathbf{x}\mathbf{W}_{xh} + \mathbf{b}_h), \quad \mathbf{h}_t = \phi(\mathbf{h}_{t-1}\mathbf{W}_{hh} + \mathbf{b}_h), \quad y = \phi(\mathbf{h}_T\mathbf{w}_{hy} + b_y)$$

$\begin{matrix} 1 \times 3 & 3 \times 2 & 1 \times 2 & 1 \times 2 & 2 \times 2 & 1 \times 2 & 1 \times 2 & 2 \times 1 & 1 \end{matrix}$

- How many learnable parameters are in this RNN? $6 + 2 + 4 + 2 + 1 = 15$
- How many learnable parameters if we increase T from 9 to 10? 15

Attention mechanism

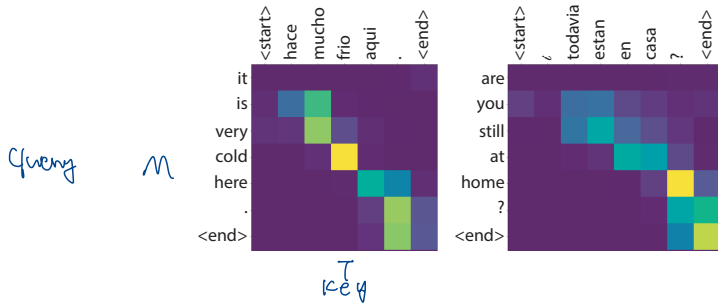
To translate T words into M words, we can therefore compute the context matrix as:

$$\mathbf{Z} = \text{Attn}(\mathbf{Q}, \mathbf{K})\mathbf{V} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D}}\right)\mathbf{V} \in \mathbb{R}^{M \times D}$$

$M \times T$ $T \times D$

where softmax function is applied over the T input words (i.e., softmax each row). We can visualize the $M \times T$ attention matrix $\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D}}\right)$ in a heatmap as below.

Attention heatmaps generated while translating Spanish to English



Module 5.4 Transformer self-attention

- Given a $N \times V$ one-hot encoded matrix $\mathbf{X}$ for a sentence of length N , we compute the embedding of the query, key, and value. The former two form the attention, which multiplies the latter to form the word embedding:

$$\text{Query: } \mathbf{Q} = \mathbf{XW}^{(q)}; \quad \text{Key: } \mathbf{K} = \mathbf{XW}^{(k)}; \quad \text{Value: } \mathbf{V} = \mathbf{XW}^{(v)}$$

$N \times V \quad V \times D \quad N \times V \quad V \times D \quad N \times V \quad V \times D'$

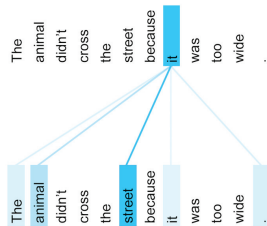
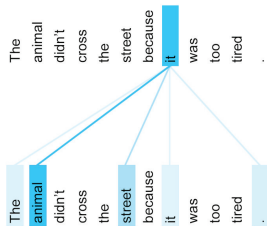
$$\text{Self-attention: } \mathbf{A} = \text{Attn}(\mathbf{Q}, \mathbf{K}); \quad \tilde{a}_{i,j} = \mathbf{q}_i \mathbf{k}_j^T / \sqrt{D}; \quad a_{i,j} = \frac{\exp(\tilde{a}_{i,j})}{\sum_j \exp(\tilde{a}_{i,j})}$$

$(N \times D) \times (D \times 1)$

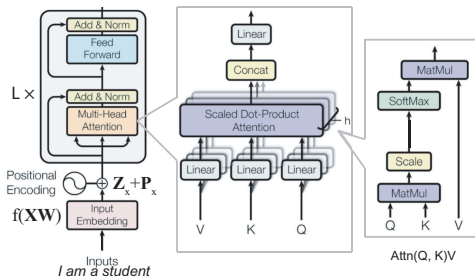
$$\text{Word embedding: } \mathbf{Z} = \mathbf{AV}$$

$N \times D \quad N \times N \quad N \times D'$

- For multi-layer embedding, $\mathbf{Z}^{\{\ell\}}$ is used as input for $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ in the next layer $\mathbf{Z}^{\{\ell+1\}}$
- The example on the right shows the word “it” attends to different words based on the context.



Putting it together: Transformer Encoder



$$Q_i = Z^{(\ell)} W_i^{(q)}, \quad K_i = Z^{(\ell)} W_i^{(k)}, \quad V_i = Z^{(\ell)} W_i^{(v)}$$

$$A_i = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{D/H}} \right) \left(a_{n,m} = \frac{\exp(q_n k_m^T)}{\sum_{m'} \exp(q_n k_{m'}^T)} \right)$$

$$Z_i = A_i V_i \quad \text{for } i \in \{1, \dots, h\}$$

$$Z^* = [Z_1, \dots, Z_h] W_h$$

$$\tilde{Z} = \text{LayerNorm}(Z^* + Z^{(\ell)})$$

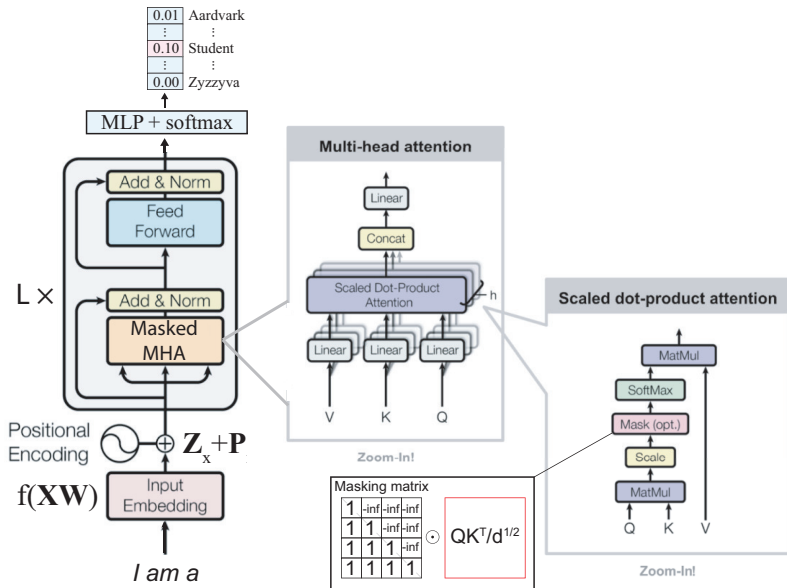
```
def EncoderBlock(X):
    Ze = LayerNorm(MHA(Q=X, K=X, V=X) + X)
    E = LayerNorm(FeedForward(Ze) + Ze)
    return E
```

```
def Encoder(X, L):
    Ze = POS(Embed(X))
    for l in range(L):
        Ze = EncoderBlock(Ze)
    return Ze
```

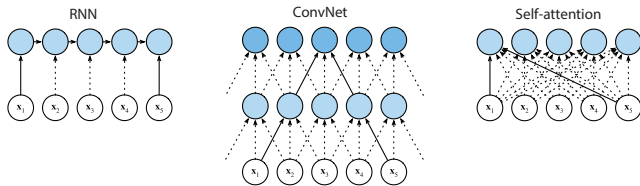
$$Z' = \text{FeedForward}(\tilde{Z}) = \begin{bmatrix} \text{ReLU}(\tilde{Z}_1 W_{FF} + b) \\ \text{ReLU}(\tilde{Z}_2 W_{FF} + b) \\ \vdots \\ \text{ReLU}(\tilde{Z}_N W_{FF} + b) \end{bmatrix}$$

$$Z^{(\ell+1)} = \text{LayerNorm}(Z' + \tilde{Z})$$

Transformer Decoder of masked self-attention for next token prediction



Comparing ConvNets, RNNs, Self-attention model



- For RNN, to map $\mathbf{X} \in \mathbb{R}^{N \times D}$ to $\mathbf{H} \in \mathbb{R}^{N \times D}$, we take the inner product of $\mathbf{x}_t$ or $\mathbf{h}_t \in \mathbb{R}^{1 \times D}$ and $\mathbf{W} \in \mathbb{R}^{D \times D}$ N times, the time complexity is $O(ND^2)$. It's inherently sequential operation that takes $O(N)$ steps. Information flows from one node to another node also takes $O(N)$.
- For 1D ConvNet with filter size K and D IO channels, the complexity is $O(KND^2)$ and we need a stack of N/K layers or $\log_K N$ if we use dilated convolution to ensure all pairs can communicate.
- For self-attention, we compute $\mathbf{X}\mathbf{X}^\top$, which takes $O(N^2D)$, which is expensive for long sequence.
- ConvNet and self-attn can be done in parallel at each layer and therefore no sequential ops.

Layer type	Complexity	Sequential ops.	Max. path length
RNN	$O(ND^2)$	$O(N)$	$O(N)$
ConvNets	$O(KND^2)$	$O(1)$	$O(\log_k N)$
Self-attention	$O(N^2D)$	$O(1)$	$O(1)$

Table: N sequence length; D : feature dimension; K : kernel size for ConvNet.

Module 7. K-means clustering (2024 practice exam)

We have the following dataset of 4 (2D) points:

$$\mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & -2 \\ -1 & 1 \\ -2 & 1 \end{bmatrix} \quad \begin{matrix} \textcircled{1} \\ \sqrt{0^2 + (-2+1)^2} < \sqrt{(1+1)^2 + (-2-1)^2} \textcircled{1} \\ \textcircled{2} \\ \textcircled{2} \end{matrix}$$

We wish to cluster the points using K-Means with $K = 2$, i.e., two clusters. We initialize our clusters with $\mathbf{c}_1 = [1, -1]$ and $\mathbf{c}_2 = [-1, 1]$.

- (2 pts) Which points belongs to which cluster initially?
- (6 pts) After the first update of the centroids, what are their new values?
- (2 pts) At convergence, what are the centroids and cluster assignments?

$$C_1 = [1, -1.5], \quad C_2 = [-1.5, 1]$$

Principal Component Analysis (PCA)

- Suppose we have an unlabelled dataset $\mathbf{X} \in \mathbb{R}^{D \times N}$ for D features and N examples.
- We would like to approximate each data point $\mathbf{x}_n \in \mathbb{R}^{D \times 1}$ by a low dimensional representation $\mathbf{z}_n \in \mathbb{R}^{K \times 1}$, where $K \leq D$.
- The variable $\mathbf{z}_n$ is known as the **latent factor**.
- The error produced by this approximation is called **reconstruction error**:

$$\begin{aligned}\mathcal{L}(\mathbf{W}) &= \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n; \mathbf{W}); \mathbf{W})\|_2^2 \\ &\triangleq \frac{1}{N} \|\mathbf{x}_n - \mathbf{W}\mathbf{z}_n\|_2^2\end{aligned}$$

$\mathbf{z}_n = \mathbf{W}^T \mathbf{x}_n$
 $K \times D \quad D \times 1$

where in the context of PCA:

- ▶ we assume a linear encoder and decoder
- ▶ $\mathbf{Z} \in \mathbb{R}^{K \times N}$ is also known as the **loading matrix**.
- ▶ $\mathbf{W} \in \mathbb{R}^{D \times K}$ is called the **basis matrix**. It is orthogonal matrix: $\mathbf{W}^T \mathbf{W} = \mathbf{I}$, which means that: for $k = k'$, $\mathbf{w}_k^T \mathbf{w}_{k'} = 1$; for $k \neq k'$, $\mathbf{w}_k^T \mathbf{w}_{k'} = 0$.

PCA learning algorithm

- The k^{th} PC is found by minimizing the reconstruction loss given the $k - 1$ PCs:

$$\mathcal{L}(\mathbf{w}_k) = \frac{1}{N} \sum_{n=1}^N \left\| \mathbf{x}_n - \sum_{s=1}^{k-1} z_{s,n} \mathbf{w}_s - z_{k,n} \mathbf{w}_k \right\|^2 = \frac{1}{N} \sum_{n=1}^N \left\| \tilde{\mathbf{x}}_n - \sum_{s=1}^k z_{s,n} \mathbf{w}_s \right\|^2$$

- We can iteratively solve $\hat{\Sigma} \mathbf{w}_k = \lambda_k \mathbf{w}_k$ for $\mathbf{w}_k$, where $\hat{\Sigma} = \frac{1}{N} \tilde{\mathbf{X}} \tilde{\mathbf{X}}^T$, $\mathbf{w}_k$ and λ_k are the k^{th} eigenvector and eigenvalue.
 $D \times N \times D$
- We can also obtain all of the PCs by the eigendecomposition:

$$\underbrace{\mathbf{X} \mathbf{X}^T}_{D \times D} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^{-1}$$

where $\mathbf{U} \in \mathbb{R}^{D \times D}$ contains all of the D eigenvectors and $\mathbf{\Lambda}$ is the diagonal matrix with the diagonal elements being the eigenvalues. Because $\mathbf{U}$ is a orthogonal matrix, $\mathbf{U}^T \mathbf{U} = \mathbf{I}$ and $\mathbf{U}^T = \mathbf{U}^{-1}$.

- More efficiently, we can compute the truncated Singular Vector Decomposition (SVD) to get only the first $K < \min(N, D)$ PCs by $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^T$.

Q & A